МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
**«Санкт-Петербургский политехнический
университет Петра Великого»**
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01_03 Технологии машинного обучения и искусственного интеллекта

Отчёт о выполнении лабораторной работы №6
Основы архитектуры ЦВМ

Выполнил:
Студент группы 5130201/50302 _____ Калинский В.И.
Принял:
К.т.н., доцент, доцент ВШТИИ и КОЦ ИИ _____ Вербова Н.М.

«____»_____ 2026 г.

Санкт-Петербург, 2026

Цель работы:

Познакомиться с системой процессорных команд и способами использования этих команд. Освоить методику изучения регистровой модели, системы команд и способов адресации с использованием экранного отладчика среды Visual Studio.

Постановка задачи (вариант 2-8):

Для группы 50302 в соответствии с методическими указаниями задан вариант 2-8: массив `int iMas[10]`, итератор `unsigned int i`. В цикле `for (i = 0; i < 9; i++)` реализуется вычисление элементов массива: если $i \geq 4$, то $iMas[i] = 13 \cdot i + 7$, иначе $iMas[i] = (i \cdot 8) \oplus 12$. В подготовительной части исследуется прямое и косвенное присваивание констант различной длины: 12, 12345, 12345678, `0x123456789ABCDE` и `0xAB12` через указатель `pI = &iMas[2]`.

1. Исходная программа и сопоставление с дизассемблером

В соответствии с шагом 1 методики в программе-шаблоне циклический фрагмент заменён кодом варианта 2-8 (листинг 1).

Листинг 1: Исходная программа (вариант 2-8, глобальные переменные)

```
int iMas[10];
int *pI;
unsigned int i;

int main() {
    iMas[3] = 12;
    iMas[4] = 12345;
    iMas[5] = 12345678;
    iMas[6] = 0x123456789ABCDE;

    pI = &iMas[2];
    *pI = 0xAB12;

    for (i = 0; i < 9; i++) {
        if (i >= 4)
            iMas[i] = 13 * i + 7;
        else
            iMas[i] = (i * 8) ^ 0xC;
    }
    return 0;
}
```

Программа оттранслирована в среде Visual Studio. В режиме отладки (окно Disassembly) получен машинный код (листинг 2) с сопоставлением инструкциям языка Си.

Листинг 2: Дизассемблированный код с сопоставлением инструкциям Си

```
; --- Пролог функции main ---
push    rbp                 ; Сохранение базового регистра кадра
push    rdi                 ; Сохранение регистра rdi
sub     rsp, 0E8h           ; Выделение стекового пространства под локальные нужды
lea     rbp, [rsp+20h]      ; Установка базового указателя кадра

; 6: iMas[3] = 12;
mov     eax, 4              ; Загрузка размера элемента int (4 байта)
imul    rax, rax, 3         ; Вычисление смещения: 3 * 4 = 12 (0x0C) байт
lea     rcx, [iMas]         ; Получение базового адреса массива iMas (07FF630CDC240h)
mov     dword ptr [rcx+rax], 0Ch ; Запись байтовой4- константы 12 по адресу rcx+
    rax

; 7: iMas[4] = 12345;
mov     eax, 4              ; Размер элемента
imul    rax, rax, 4         ; Смещение элемента: 4 * 4 = 16 (0x10) байт
lea     rcx, [iMas]         ; Базовый адрес массива iMas
mov     dword ptr [rcx+rax], 3039h ; Запись константы 12345 (0x3039)

; 8: iMas[5] = 12345678;
mov     eax, 4              ; Размер элемента
imul    rax, rax, 5         ; Смещение элемента: 5 * 4 = 20 (0x14) байт
lea     rcx, [iMas]         ; Базовый адрес массива iMas
mov     dword ptr [rcx+rax], 0BC614Eh ; Запись константы 12345678 (0x00BC614E)

; 9: iMas[6] = 0x123456789ABCDE;
mov     eax, 4              ; Размер элемента
imul    rax, rax, 6         ; Смещение элемента: 6 * 4 = 24 (0x18) байта
lea     rcx, [iMas]         ; Базовый адрес массива iMas
mov     dword ptr [rcx+rax], 789ABCDEh ; Усечение до 32 бит: запись младших 4 байт

; 11: pI = &iMas[2];
mov     eax, 4              ; Размер элемента
imul    rax, rax, 2         ; Смещение для индекса 2: 2 * 4 = 8 байт
lea     rcx, [iMas]         ; Базовый адрес массива iMas
add     rcx, rax            ; rcx = адрес элемента iMas[2]
mov     rax, rcx            ; Перенос адреса в rax
mov     qword ptr [pI], rax ; Запись байтового8- адреса в указатель pI

; 12: *pI = 0xAB12;
mov     rax, qword ptr [pI] ; Считывание адреса из pI в регистр rax
```

```
41 mov     dword ptr [rax], 0AB12h ; Косвенная запись константы 0xAB12 по адресу в rax
42
43 ; 14: for (i = 0; i < 9; i++) {
44 mov     dword ptr [i], 0 ; i = 0 инициализация( счетчика цикла)
45 jmp     main+0BCh        ; Безусловный переход к проверке условия цикла
46
47 main+0CEh:               ; Шаг цикла (i++)
48 mov     eax, dword ptr [i] ; Загрузка текущего i в eax
49 inc     eax                ; Инкремент значения (i + 1)
50 mov     dword ptr [i], eax ; Сохранение обновленного i обратно в память
51
52 main+0BCh:               ; Проверка условия продолжения цикла (i < 9)
53 cmp     dword ptr [i], 9 ; Сравнение i с константой 9
54 jae     main+108h        ; Если i >= 9, выход из цикла беззнаковый( переход)
55
56 ; 15: if (i >= 4)
57 cmp     dword ptr [i], 4 ; Сравнение счетчика i с числом 4
58 jb      main+0EAh        ; Если i < 4, переход в ветку else беззнаковый( переход)
59
60 ; 16: iMas[i] = 13 * i + 7;
61 imul    eax, dword ptr [i], 0Dh ; eax = i * 13 (0x0D)
62 add     eax, 7           ; eax = 13 * i + 7
63 mov     ecx, dword ptr [i] ; Загрузка индекса i в ecx
64 lea     rdx, [iMas]      ; Базовый адрес начала массива в rdx
65 mov     dword ptr [rdx+rcx*4], eax ; Запись в массив: База + Индекс * 4
66 jmp     main+106h        ; Обход ветки else
67
68 ; 18: else iMas[i] = (i * 8) ^ 0xC;
69 main+0EAh:
70 mov     eax, dword ptr [i] ; Загрузка i в eax
71 shl     eax, 3           ; Умножение на 8 сдвигом влево на 3 бита
72 xor     eax, 0Ch         ; Побитовое исключающее ИЛИ с константой 12 (0x0C)
73 mov     ecx, dword ptr [i] ; Загрузка индекса i в ecx
74 lea     rdx, [iMas]      ; Базовый адрес начала массива в rdx
75 mov     dword ptr [rdx+rcx*4], eax ; Запись в массив: База + Индекс * 4
76
77 main+106h:               ; Конец тела цикла
78 jmp     main+0CEh        ; Переход к следующей итерации на( шаг i++)
79
80 ; 21: return 0;
81 main+108h:
82 xor     eax, eax         ; Код возврата 0
83 lea     rsp, [rbp+0C8h]  ; Восстановление стека
84 pop     rdi              ; Восстановление сохранённых регистров
85 pop     rbp              ; Восстановление базового указателя
86 ret                      ; Возврат из функции
```

87

2. Список переменных и карта памяти

Переменные `iMas`, `pI`, `i` объявлены вне функций (глобально) и размещены компилятором в статической памяти данных. Их адреса постоянны и зафиксированы отладчиком (таблица 1).

Таблица 1: Список переменных и карта памяти программы

Имя	**Класс**	**Тип**	**Размер**	**Адрес в памяти**
`iMas[0..9]`	глобальная	`int[10]`	40 байт	`0x07FF630CDC240` – `0x07FF630CDC267`
`pI`	глобальная	`int*`	8 байт	`0x07FF630CDC268` – `0x07FF630CDC26F`
`i`	глобальная	`unsigned int`	4 байта	`0x07FF630CDC270` – `0x07FF630CDC273`

Переменные расположены последовательно: массив `iMas` занимает 40 байт (28_{16}), следом 8-байтовый указатель `pI`, затем 4-байтовый счетчик `i`.

3. Анализ команд обращения к переменным

В таблице 2 систематизированы все команды, обращающиеся к переменным программы, с указанием числа операндов и способов их адресации.

Таблица 2: Команды обращения к переменным программы

Команда ассемблера	**Перем.**	**Оп.**	**Операнд 1 (приёмник)**	**Операнд 2 (источник)**
`mov [rcx+rax], 0Ch`	`iMas[3]`	2	Базово-индексная	Непосредственная
`mov [rcx+rax], 3039h`	`iMas[4]`	2	Базово-индексная	Непосредственная
`mov [rcx+rax], 0BC614Eh`	`iMas[5]`	2	Базово-индексная	Непосредственная
`mov [rcx+rax], 789ABCDEh`	`iMas[6]`	2	Базово-индексная	Непосредственная
`mov qword ptr [pI], rax`	`pI`	2	Прямая к памяти	Регистровая
`mov rax, qword ptr [pI]`	`pI`	2	Регистровая	Прямая к памяти
`mov dword ptr [rax], 0AB12h`	`iMas[2]`	2	Косвенно-регистровая	Непосредственная
`mov dword ptr [i], 0`	`i`	2	Прямая к памяти	Непосредственная
`mov eax, dword ptr [i]`	`i`	2	Регистровая	Прямая к памяти
`mov dword ptr [i], eax`	`i`	2	Прямая к памяти	Регистровая
`cmp dword ptr [i], 9`	`i`	2	Прямая к памяти	Непосредственная
`cmp dword ptr [i], 4`	`i`	2	Прямая к памяти	Непосредственная
`imul eax, dword ptr [i], 0Dh`	`i`	3	Регистровая	Прямая и непоср.
`mov ecx, dword ptr [i]`	`i`	2	Регистровая	Прямая к памяти
`mov [rdx+rcx*4], eax`	`iMas[i]`	2	Масштабно-индексная	Регистровая

4. Многокомпонентная адресация данных

Многокомпонентная масштабно-индексная адресация использована в команде записи в массив в теле цикла:

`mov dword ptr [rdx+rcx*4], eax` (машинный код: `89 04 8A`)

Значения отдельных компонент исполнительного адреса:

- **База (Base):** регистр `rdx` — содержит базовый адрес начала массива `iMas` (`0x07FF630CDC240`);
- **Индекс (Index):** регистр `rcx` — содержит значение переменной `i` ($0 \leq i < 9$);
- **Масштаб (Scale):** коэффициент 4 — соответствует размеру типа `sizeof(in = 4` байта;
- **Смещение (Displacement):** отсутствует (равно 0).

Исполнительный адрес вычисляется аппаратно за один такт: $EA = rdx + rcx \cdot 4$.

5. Способы адресации в командах перехода

Во всех командах ветвления компилятор применил **относительную адресацию** (`rel8`): адрес перехода не хранится целиком, а задаётся знаковым смещением относительно счётчика команд следующей инструкции (Target = IP + offset):

- `EB 0E (jmp main+0BCh)`: безусловный короткий переход на проверку условия;
- `73 43 (jae main+108h)`: беззнаковый переход при выходе из цикла ($i \geq 9$);
- `72 1C (jb main+0EAh)`: беззнаковый переход в ветку `else` при $i < 4$ (по флагу $CF = 1$);
- `EB 1C (jmp main+106h)`: безусловный переход в обход ветки `else`;
- `EB A6 (jmp main+0CEh)`: безусловный переход на шаг цикла со смещением `0xA6` (-90).

6. Модификация программы: доступ через указатель и локальные переменные

В соответствии с шагом 10 методики доступ к массиву изменён на последовательное обращение через указатель `pMas` (листинг 3).

Листинг 3: Модифицированная программа с доступом через указатель

```
int iMas[10];
int *pI;
unsigned int i;

int main() {
    iMas[3] = 12;
    iMas[4] = 12345;
    iMas[5] = 12345678;
    iMas[6] = 0x123456789ABCDE;

    pI = &iMas[2];
    *pI = 0xAB12;

    int *pMas = iMas;
    for (i = 0; i < 9; i++, pMas++) {
        if (i >= 4)
            *pMas = 13 * i + 7;
        else
            *pMas = (i * 8) ^ 0xC;
    }
    return 0;
}

```

Дизассемблированный фрагмент модифицированного цикла:

Листинг 4: Дизассемблерный код цикла при работе через указатель

```
; Инициализация указателя перед циклом:
lea     rax, [iMas]               ; Получение адреса начала массива
mov     qword ptr [pMas], rax     ; Сохранение адреса в локальный указатель pMas

; Шаг цикла (i++, pMas++):
mov     eax, dword ptr [i]        ; Загрузка счетчика i
inc     eax                       ; i++
mov     dword ptr [i], eax        ; Сохранение обновленного i
mov     rax, qword ptr [pMas]     ; Загрузка адреса из pMas
add     rax, 4                    ; Инкремент адреса на 4 байта размер( int)
mov     qword ptr [pMas], rax     ; Сохранение нового адреса в pMas

; Запись в массив в ветках then и else (*pMas = ...):
mov     rcx, qword ptr [pMas]     ; Загрузка адреса текущего элемента из pMas в rcx
mov     dword ptr [rcx], eax      ; Простая косвеннорегистровая- адресация [rcx]!
```

Сравнение способов адресации:

1. **Индекс против указателя:** При доступе по индексу процессор на каждом шаге вычисляет формулу $rdx+rcx\cdot4$ командой `mov [rdx+rcx*4], eax` (машинный код `89 04 8A`, 3 байта, содержит SIB-байт). При доступе через указатель запись выполняется короткой командой косвенно-регистровой адресации `mov [rcx], eax` (код `89 01`, 2 байта, без SIB-байта), а шаг к следующему элементу сводится к простому сложению `add rax, 4`.
2. **Глобальное против локального размещения:** Глобальные переменные размещены в статической памяти и адресуются по фиксированным смещениям. Локальные переменные размещаются в стеке и адресуются со смещением относительно регистра базы кадра: `[rbp-offset]`.

Выводы:

В ходе лабораторной работы изучена система команд и способы адресации процессоров архитектуры x86. Проанализировано размещение глобальных данных в памяти, составлена карта памяти. Установлено, что при обращении к элементам массива по индексу используется многокомпонентная масштабно-индексная адресация с масштабом 4 (размер типа `int`). Замена индексации на указатель упрощает команду записи до короткой косвенно-регистровой (`mov [rcx], eax`) без вычисления адреса на каждой итерации. Проверка условий беззнаковых переменных (`unsigned`) транслируется в беззнаковые переходы (`jb`, `jae`), а умножение на степень двойки ($i\cdot8$) компилятор эффективно заменяет побитовым сдвигом влево (`shl eax, 3`).